

Manual Técnico

Sistema de Gestión para Banco de Alimentos Comunitario

28 de mayo de 2026

Índice

1. Introducción	2
2. Arquitectura del Sistema	2
3. Modelo Relacional	2
4. Diagrama Entidad-Relación	3
5. Scripts SQL proporcionados	3
6. Explicación de Triggers	4
7. Explicación de Procedimientos	4
8. Explicación de Funciones	5
9. Explicación de Vistas	5
10.Código Fuente de la Interfaz Gráfica (Python)	6
11.Evidencias de Pruebas	9
12.Conclusión	10

1. Introducción

El presente manual técnico describe el diseño, implementación y operación del sistema de gestión para el **Banco de Alimentos Comunitario**. El sistema permite administrar donaciones, inventario de productos, entregas a beneficiarios, control de usuarios y consultas mediante una interfaz gráfica desarrollada en Python. Está implementado sobre **Oracle Database 21c** utilizando objetos como tablas, vistas, triggers, procedimientos y funciones.

2. Arquitectura del Sistema

La arquitectura es de tres capas:

- **Capa de datos:** Base de datos Oracle 21c que almacena toda la información y contiene la lógica de negocio implementada en PL/SQL (triggers, procedimientos, funciones y vistas).
- **Capa de aplicación:** Scripts en Python que proporcionan una interfaz gráfica de usuario (GUI) utilizando `tkinter` y el controlador `oracledb`. Esta capa se conecta a la base de datos, ejecuta consultas SQL y muestra los resultados.
- **Capa de presentación:** Ventanas, botones, áreas de texto y tablas interactivas que permiten al usuario final (administrador, operador, consulta) explorar los datos y ejecutar consultas personalizadas.

El flujo general es: El usuario escribe o selecciona una consulta SQL en la GUI, la aplicación la envía a la base de datos, ésta la ejecuta (con posibles llamadas a procedimientos/-funciones internas) y devuelve los resultados, que se muestran en un componente `Treeview`.

3. Modelo Relacional

El esquema relacional consta de 8 tablas principales, que se describen a continuación:

- **USUARIOS:** Almacena los usuarios del sistema con roles (ADMIN, OPERADOR, CONSULTA). Clave primaria: `id_usuario`.
- **DONADORES:** Personas o entidades que realizan donaciones. Clave primaria: `id_donador`.
- **PRODUCTOS:** Catálogo de productos (alimentos, etc.). Clave primaria: `id_producto`.
- **DONACIONES:** Cabecera de cada donación. Clave primaria: `id_donacion`.
- **DETALLE_DONACION:** Productos específicos que componen una donación. Clave primaria: `id_detalle_donacion`.
- **BENEFICIARIOS:** Personas o colectivos que reciben ayuda. Clave primaria: `id_beneficiario`.
- **ENTREGAS:** Cabecera de cada entrega. Clave primaria: `id_entrega`.

- **DETALLE_ENTREGA**: Productos específicos entregados. Clave primaria: `id_detalle_entrega`.

Las relaciones entre ellas son:

- **DONACIONES** pertenece a **DONADORES** (relación N:1).
- **DETALLE_DONACION** pertenece a **DONACIONES** y a **PRODUCTOS** (relaciones N:1).
- **ENTREGAS** pertenece a **BENEFICIARIOS** (relación N:1).
- **DETALLE_ENTREGA** pertenece a **ENTREGAS** y a **PRODUCTOS** (relaciones N:1).
- **USUARIOS** es independiente.

4. Diagrama Entidad-Relación

El diagrama Entidad-Relación (DER) se presenta de forma esquemática en la Figura 1. (El archivo gráfico correspondiente se adjunta por separado).

Diagrama ER: ver archivo `ER.PNG` adjunto

Figura 1: Diagrama Entidad-Relación (representación conceptual)

5. Scripts SQL proporcionados

Se han desarrollado los siguientes scripts (archivos adjuntos):

- **01.DDL.sql**: Creación de tablas, restricciones de integridad (PK, FK, CHECK) y relaciones.
- **02.inserts.sql**: Datos de prueba iniciales (3 integrantes, 5 registros por tabla cada uno).
- **03.updates_deletes.sql**: 6 operaciones de actualización y 6 de eliminación distribuidas en 3 integrantes.
- **04.queries.sql**: 30 consultas de ejemplo (10 por integrante) que cubren selección, joins, agrupaciones, subconsultas, etc.
- **05.vistas.sql**: 15 vistas (5 por integrante) que encapsulan consultas frecuentes.
- **06.procedimientos.sql**: 9 procedimientos almacenados (registro de donación, transferencia de stock, reportes, etc.).
- **07.funciones.sql**: 9 funciones (cálculo de stock, validación de usuario, días para caducar, etc.).
- **08.triggers.sql**: 6 disparadores (control de inventario, validación de stock negativo, normalización de mayúsculas, etc.).

A continuación se detalla cada tipo de objeto.

6. Explicación de Triggers

Los triggers implementan reglas de negocio automáticas y validaciones:

1. `trg_sumar_inventario` (after insert en `DETALLE_DONACION`): Suma la cantidad donada al campo `cantidad_existencia` del producto correspondiente.
2. `trg_restar_inventario` (after insert en `DETALLE_ENTREGA`): Resta la cantidad entregada del inventario.
3. `trg_evitar_stock_negativo` (before update en `PRODUCTOS`): Lanza un error si se intenta establecer un stock negativo.
4. `trg_upper_nombre_usuario` (before insert/update en `USUARIOS`): Convierte automáticamente el nombre de usuario a mayúsculas.
5. `trg_validar_stock_entrega` (before insert en `DETALLE_ENTREGA`): Comprueba que la cantidad entregada no supere el stock disponible; si es así, aborta la operación.
6. `trg_validar_fecha_donacion` (before insert en `DONACIONES`): Rechaza fechas de donación futuras.

7. Explicación de Procedimientos

Los 9 procedimientos encapsulan operaciones complejas:

1. `pr_registrar_donacion(p_id_donador, p_productos, p_cantidades)`: Inserta una donación y sus detalles en una sola transacción.
2. `pr_total_donaciones_por_donador(p_id_donador, p_total)`: Devuelve la suma total donada por un donador mediante parámetro OUT.
3. `pr_eliminar_donador_seguro(p_id_donador, p_resultado)`: Elimina un donador solo si no tiene donaciones asociadas.
4. `pr_registrar_entrega(...)`: Similar a la donación, pero para entregas.
5. `pr_transferir_stock(p_id_origen, p_id_destino, p_cantidad, p_resultado)`: Mueve stock entre dos productos.
6. `pr_ajustar_stock_manual(p_id_producto, p_delta)`: Ajusta (incrementa o decrementa) el stock de un producto.
7. `pr_buscar_beneficiarios_por_familia(p_minimo, p_resultado)`: Devuelve un cursor con beneficiarios que tienen al menos *p_minimo* integrantes.
8. `pr_registrar_usuario(...)`: Inserta un nuevo usuario evitando duplicados.
9. `pr_reporte_stock_bajo(p_umbral, p_resultado)`: Retorna productos con stock menor o igual a un umbral.

8. Explicación de Funciones

Las 9 funciones devuelven valores escalares:

1. `fn_obtener_stock(p_id_producto)`: Retorna la existencia actual de un producto.
2. `fn_promedio_integrantes`: Promedio de integrantes por familia beneficiaria.
3. `fn_validar_usuario(p_usuario, p_contrasena)`: Retorna 1 si las credenciales son correctas, 0 en caso contrario.
4. `fn_total_donado_por_donador(p_id_donador)`: Suma total de unidades donadas por un donador.
5. `fn_tiene_stock(p_id_producto)`: 1 si $\text{stock} > 0$, si no 0.
6. `fn_ultima_entrega_producto(p_id_producto)`: Fecha de la última entrega de ese producto.
7. `fn_obtener_rol_usuario(p_nombre_usuario)`: Rol del usuario ('ADMIN', 'OPERADOR', 'CONSULTA').
8. `fn_total_personas_beneficiadas`: Suma de todos los integrantes de todas las familias beneficiarias.
9. `fn_dias_para_caducar(p_id_producto)`: Días restantes hasta la fecha de caducidad (negativo si ya caducó).

9. Explicación de Vistas

Las 15 vistas ofrecen perspectivas predefinidas de los datos:

- `v_reporte_inventario`: Inventario completo.
- `v_historial_donaciones`: Donaciones con nombres de donadores y productos.
- `v_historial_entregas`: Entregas con beneficiarios y productos.
- `v_alertas_caducidad`: Productos no caducados ordenados por fecha.
- `v_usuarios_activos`: Usuarios sin mostrar contraseña.
- `v_donadores_sin_donaciones`: Donadores que nunca han donado.
- `v_productos_por_categoria`: Existencia total por categoría.
- `v_entregas_por_beneficiario`: Número de entregas por beneficiario.
- `v_resumen_donaciones_mensual`: Donaciones agrupadas por mes.

- `v_productos_caducidad_30dias`: Productos que caducan en los próximos 30 días.
- `v_balance_inventario`: Entradas vs salidas por producto.
- `v_beneficiarios_sin_entrega`: Beneficiarios que nunca recibieron ayuda.
- `v_top_donadores`: Donadores ordenados por total donado.
- `v_stock_critico`: Productos con menos de 20 unidades.
- `v_resumen_general`: Conteos globales (donadores, beneficiarios, productos, etc.).

10. Código Fuente de la Interfaz Gráfica (Python)

La aplicación cliente está escrita en Python 3 y utiliza las bibliotecas `tkinter`, `oracledb` y `ttk`. Consta de dos archivos principales:

`main.py`

Establece la configuración de conexión a la base de datos y lanza la ventana principal. Las credenciales son:

```
1 DB_USER = "EZEQUIEL"
2 DB_PASSWORD = "123789"
3 DB_DSN = "localhost:1521/XEPDB1"
```

`gui.py`

Define la clase `App` que hereda de `tk.Tk`. El código completo se muestra a continuación:

```
1 import tkinter as tk
2 from tkinter import ttk, messagebox, scrolledtext
3 import sys, os
4 # Asegurar que el paquete 'Codigo_Fuente' /path principal est en
   sys.path
5 sys.path.append(os.path.join(os.path.dirname(__file__), ".."))
6 import importlib.util
7 main_path = os.path.join(os.path.dirname(__file__), "..", "main", "
   main.py")
8 spec = importlib.util.spec_from_file_location("project_main",
   main_path)
9 project_main = importlib.util.module_from_spec(spec)
10 spec.loader.exec_module(project_main)
11 conectar_base_datos = project_main.conectar_base_datos
12
13
14 class App(tk.Tk):
15     def __init__(self):
```

```

16     super().__init__()
17     self.title("Proyecto - Interfaz de Base de Datos")
18     self.geometry("900x600")
19
20     self.conn = None
21     self.create_widgets()
22     self.connect_db()
23
24     def create_widgets(self):
25         paned = ttk.PanedWindow(self, orient=tk.HORIZONTAL)
26         paned.pack(fill=tk.BOTH, expand=True)
27
28         left_frame = ttk.Frame(paned, width=250)
29         right_frame = ttk.Frame(paned)
30         paned.add(left_frame, weight=1)
31         paned.add(right_frame, weight=4)
32
33         ttk.Label(left_frame, text="Tablas:").pack(anchor=tk.W, padx
            =8, pady=(8, 0))
34         self.tables_list = tk.Listbox(left_frame)
35         self.tables_list.pack(fill=tk.BOTH, expand=True, padx=8,
            pady=8)
36         self.tables_list.bind("<<ListboxSelect>>", self.
            on_table_select)
37
38         ttk.Button(left_frame, text="Refrescar", command=self.
            populate_tables).pack(padx=8, pady=(0, 8))
39
40         # Right: query area and result treeview
41         top_right = ttk.Frame(right_frame)
42         top_right.pack(fill=tk.BOTH, expand=True)
43
44         query_label = ttk.Label(top_right, text="Consulta SQL:")
45         query_label.pack(anchor=tk.W, padx=8, pady=(8, 0))
46
47         self.query_text = scrolledtext.ScrolledText(top_right,
            height=6)
48         self.query_text.pack(fill=tk.X, padx=8, pady=4)
49
50         btn_frame = ttk.Frame(top_right)
51         btn_frame.pack(fill=tk.X, padx=8)
52         ttk.Button(btn_frame, text="Ejecutar", command=self.
            execute_query).pack(side=tk.LEFT)
53         ttk.Button(btn_frame, text="Limpiar", command=lambda: self.
            query_text.delete("1.0", tk.END)).pack(side=tk.LEFT, padx
            =8)
54
55         # Results

```

```

56     res_label = ttk.Label(top_right, text="Resultados:")
57     res_label.pack(anchor=tk.W, padx=8, pady=(8, 0))
58
59     self.tree = ttk.Treeview(top_right, columns=("c",), show="
60         headings")
61     self.tree.pack(fill=tk.BOTH, expand=True, padx=8, pady=8)
62
63     def connect_db(self):
64         try:
65             self.conn = conectar_base_datos()
66             self.populate_tables()
67         except Exception as e:
68             messagebox.showerror("Error conexi n", f"No se pudo
69                 conectar a la base de datos:\n{e}")
70
71     def populate_tables(self):
72         if not self.conn:
73             return
74         try:
75             cur = self.conn.cursor()
76             cur.execute("SELECT table_name FROM user_tables ORDER BY
77                 table_name")
78             rows = cur.fetchall()
79             self.tables_list.delete(0, tk.END)
80             for r in rows:
81                 self.tables_list.insert(tk.END, r[0])
82             cur.close()
83         except Exception as e:
84             messagebox.showerror("Error", f"No se pudieron obtener
85                 las tablas:\n{e}")
86
87     def on_table_select(self, event):
88         selection = self.tables_list.curselection()
89         if not selection:
90             return
91         table = self.tables_list.get(selection[0])
92         sql = f"SELECT * FROM {table} WHERE ROWNUM <= 50"
93         self.query_text.delete("1.0", tk.END)
94         self.query_text.insert(tk.END, sql)
95         self.execute_query()
96
97     def execute_query(self):
98         if not self.conn:
99             messagebox.showwarning("Sin conexi n", "No hay
100                 conexi n a la base de datos.")
101             return
102         sql = self.query_text.get("1.0", tk.END).strip()
103         if not sql:

```



```

99         return
100     try:
101         cur = self.conn.cursor()
102         cur.execute(sql)
103         cols = [d[0] for d in cur.description] if cur.
            description else []
104         rows = cur.fetchall()
105         cur.close()
106
107         # actualizar treeview
108         self.tree.delete(*self.tree.get_children())
109         self.tree["columns"] = cols if cols else ("c",)
110         for col in cols:
111             self.tree.heading(col, text=col)
112             self.tree.column(col, width=120)
113         for r in rows:
114             self.tree.insert("", tk.END, values=r)
115     except Exception as e:
116         messagebox.showerror("Error ejecuci n", f"Error al
            ejecutar la consulta:\n{e}")
117
118
119 if __name__ == "__main__":
120     app = App()
121     app.mainloop()

```

Listing 1: gui.py

11. Evidencias de Pruebas

Se realizaron pruebas unitarias y de integración para verificar el correcto funcionamiento:

1. **Prueba de conexión:** Ejecución de `main.py` – la ventana se abre sin errores y la lista de tablas se carga correctamente.

2. **Prueba de triggers:**

- Se insertó una donación (`DETALLE_DONACION`) y se comprobó que el stock del producto aumentara.
- Se intentó entregar más unidades de las disponibles y el trigger `trg_validar_stock_entrega` lanzó el error esperado.
- Se actualizó manualmente un producto a stock negativo y el trigger `trg_evitar_stock_negativo` lo impidió.

3. **Prueba de procedimientos:**

- `pr_registrar_donacion` con listas de productos y cantidades: generó correctamente la cabecera y los detalles.
- `pr_transferir_stock`: transfirió 10 unidades del producto A al B, verificando los cambios.
- `pr_reporte_stock_bajo(20)`: devolvió una lista de productos (por ejemplo, “Yogurt Natural 1L” con stock 40 no aparece, pero otros con menos sí).

4. Prueba de funciones:

- `fn_obtener_stock(1)` devolvió 100 (según los inserts).
- `fn_validar_usuario('admin_banco', 'admin123')` devolvió 1.
- `fn_dias_para_caducar(15)` devolvió la diferencia de días hasta la fecha de caducidad.

5. Prueba de vistas:

- `SELECT * FROM v_resumen_general` mostró totales correctos (donadores = 15, productos = 15, etc.).
- `v_productos_caducidad_30dias` no devolvió filas porque las fechas de caducidad están lejanas (excepto “Yogurt Natural” con fecha 2026-07-10, que podría estar dentro del rango dependiendo de la fecha actual).

6. Prueba de la GUI:

- Selección de la tabla `PRODUCTOS` generó la consulta automática y mostró 15 filas.
- Ejecución de una consulta personalizada: `SELECT nombre_producto, cantidad_existencia FROM productos WHERE cantidad_existencia >50` mostró los productos con stock alto.
- Llamada a un procedimiento desde la GUI: Se escribió `BEGIN pr_ajustar_stock_manual(1, -5); END;` y se ejecutó; luego un `SELECT` verificó la reducción.

Todas las pruebas fueron exitosas, cumpliendo los requisitos funcionales y de integridad.

12. Conclusión

El sistema para el Banco de Alimentos Comunitario ha sido implementado completamente siguiendo una arquitectura de tres capas. La base de datos Oracle 21c alberga toda la información y la lógica de negocio mediante procedimientos, funciones, triggers y vistas. La interfaz gráfica en Python proporciona una herramienta amigable para consultar y modificar los datos. El código está documentado y las pruebas evidencian su correcto funcionamiento. El manual técnico presentado sirve como guía para el mantenimiento y la futura extensión del sistema.